

1. What is the Common Intermediate Language?

Brief summary: The Common Intermediate Language is a programming language that all .NET-compatible languages like C#, Visual Basic, or F# get compiled to.

The **Common Intermediate Language** (sometimes, for short, referred to as the Intermediate Language) is a programming language. When the source code written in .NET-compatible languages like C#, Visual Basic, or F# gets compiled, it is transformed into code written in Common Intermediate Language. When the application is started, the Common Language Runtime's **JIT Compiler** translates the CIL code to binary code (JIT stands for "Just-In-Time", which means that a particular piece of code will be translated from CIL to binary code just before it is executed for the first time).

Let's write a simple C# code and see what it looks like after being translated to CIL:

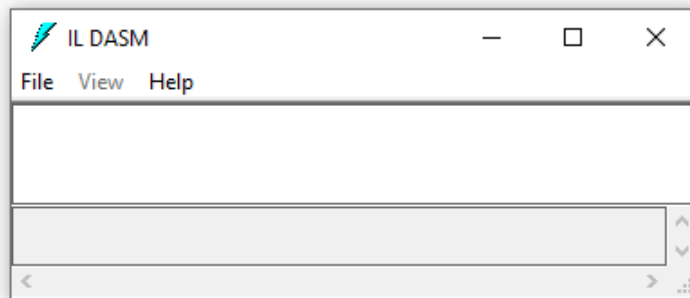
```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello! What's your name?");
        var name = Console.ReadLine();

        Console.WriteLine($"Nice to meet you, {name}. How old are you?");

        var ageAsText = Console.ReadLine();

        if(int.TryParse(ageAsText, out int age))
        {
            Console.WriteLine($"That young, only {age}?");
        }
        else
        {
            Console.WriteLine("Sorry, I didn't get that.");
        }
        Console.WriteLine("Well, it was nice to meet you! Bye, bye!");
        Console.ReadKey();
    }
}
```

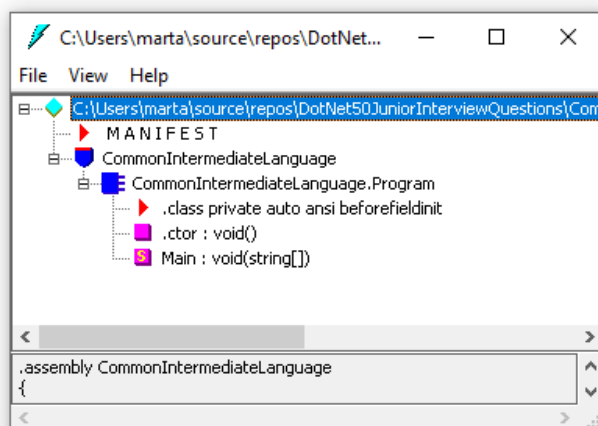
To see the CIL code, we must first make sure to build the solution in the Visual Studio. Then we are going to use **ildasm** to view the CIL code. Ildasm is the Intermediate Language Disassembler, and it gets installed when you install .NET on your machine. On my machine it got installed in C:\Program Files (x86)\Microsoft SDKs\Windows\v10.0A\bin\NETFX 4.8 Tools\ildasm.exe. Let's run this tool:



Now, we must find the *.dll for which we want to see the CIL code. The simplest solution is to right-click the project in the Visual Studio and then select "Open Folder in File Explorer". Then, we must go to the output folder. In my case it is /bin/Debug/net5.0. There I can find the *.dll that was built by the Visual Studio:

ref	19.06.2021 17:33
CommonIntermediateLanguage.deps	19.06.2021 17:33
CommonIntermediateLanguage.dll	19.06.2021 17:33
CommonIntermediateLanguage	19.06.2021 17:33
CommonIntermediateLanguage.pdb	19.06.2021 17:33
CommonIntermediateLanguage.runtime...	19.06.2021 17:33
CommonIntermediateLanguage.runtime...	19.06.2021 17:33

We can simply drag and drop it to Ildasm:



All right, let's see how the Main method looks in CIL code. Let's double-click on the Main method in Ildasm. This is what we will see:

```
CommonIntermediateLanguage.Program::Main : void(string[])
Find Find Next

.method private hidebysig static void Main(string[] args) cil managed
{
    .entrypoint
    // Code size      115 (0x73)
    .maxstack 3
    .locals init (string U_0,
                  string U_1,
                  int32 U_2,
                  bool U_3)
    IL_0000: nop
    IL_0001: ldstr      "Hello! What's your name\?"
    IL_0006: call       void [System.Console]System.Console::WriteLine(string)
    IL_000b: nop
    IL_000c: call       string [System.Console]System.Console::ReadLine()
    IL_0011: stloc.0
    IL_0012: ldstr      "Nice to meet you, "
    IL_0017: ldloc.0
    IL_0018: ldstr      ". How old are you\?"
    IL_001d: call       string [System.Runtime]System.String::Concat(string,
                                                             string,
                                                             string)
    IL_0022: call       void [System.Console]System.Console::WriteLine(string)
    IL_0027: nop
    IL_0028: call       string [System.Console]System.Console::ReadLine()
    IL_002d: stloc.1
    IL_002e: ldloc.1
    IL_002f: ldloca.s  U_2
    IL_0031: call       bool [System.Runtime]System.Int32::TryParse(string,
                                                            int32&)
    IL_0036: stloc.3
    IL_0037: ldloc.3
    IL_0038: brfalse.s IL_0054
    IL_003a: nop
    IL_003b: ldstr      "That young, only {0}\?"
    IL_0040: ldloc.2
    IL_0041: box       [System.Runtime]System.Int32
    IL_0046: call       string [System.Runtime]System.String::Format(string,
                                                                object)
    IL_004b: call       void [System.Console]System.Console::WriteLine(string)
    IL_0050: nop
    IL_0051: nop
    IL_0052: br.s     IL_0061
    IL_0054: nop
    IL_0055: ldstr      "Sorry, I didn't get that."
    IL_005a: call       void [System.Console]System.Console::WriteLine(string)
    IL_005f: nop
    IL_0060: nop
    IL_0061: ldstr      "It was nice to meet you!"
    IL_0066: call       void [System.Console]System.Console::WriteLine(string)
    IL_006b: nop
    IL_006c: call       valuetype [System.Console]System.ConsoleKeyInfo [System.Console]System.Console::ReadKey()
    IL_0071: pop
    IL_0072: ret
} // end of method Program::Main
```

This might not be the most beautiful programming language you've ever seen, but on the other hand, it is not completely unreadable, as one could expect.

Remember - all .NET compatible languages, not only C#, get compiled to the CIL. That enables communication between, for example, a C# and an F# libraries. For example, we can have a C# class derived from an F# class exactly because they both get compiled to the same programming language - the CIL.

Tip: other interview questions on this topic:

- **"How is it possible that a C# class can derive from, for example, an F# class?"**
It is possible because both those languages are .NET compatible and they get compiled to the Common Intermediate Language.
- **"Does C# compiler compile C# source code directly to binary code?"**
No, it compiles it to the Intermediate Language, which is compiled to binary code by the Just-In-Time compiler in runtime.
- **"How can you see the CIL code a project got compiled to?"**
*Some tools can decompile a *.dll file and read the CIL code. One of those tools is Ildasm.*
- **"What is the Just-In-Time compiler?"**
Just-In-Time compiler is a feature of the Common Language Runtime (CLR), which translates the Common Intermediate Language (CIL) code to binary code during the program execution.